SHOULD BE REPLACED ON REQUIRED TITLE PAGE

Instruction

1. Open needed docx template (folder "title"/<your department or bach if bachelor student>.docx).

2. Put Thesis topic, supervisor's and your name in appropriate places on both English and Russian languages.

3. Put current year (last row).

4. Convert it to "title.pdf," replace the existing one in the root folder.

Contents

List of Tables

List of Figures

Abstract

Container orchestration platforms such as Kubernetes and HashiCorp Nomad have become foundational infrastructure for managing distributed workloads, yet they impose substantial operational overhead and require specialized expertise to maintain. Their predominant push-based scheduling model introduces fault-tolerance challenges, including stale delivery states and potential workload duplication under partial failures. This thesis explores a pull-based coordination model in which worker nodes independently poll a lightweight control plane for available workloads and acquire exclusive leases to claim ownership, delegating coordination to a relational database rather than implementing custom scheduling algorithms, consensus protocols, or persistent agent state. A prototype is implemented and evaluated on distributed infrastructure across scenarios covering concurrent scheduling, automatic failure recovery, and network routing. The results demonstrate that the pull-based model achieves reliable workload placement and fault recovery while requiring a fraction of the resources consumed by existing platforms, suggesting that a simpler approach to orchestration can remain practical without sacrificing core reliability guarantees.

Chapter 1

Introduction

Modern cloud-based software systems are increasingly expected to be reliable, scalable, and highly available. As organizations adopt cloud-native practices and deploy distributed applications, the need for platforms that enable continuous monitoring, maintenance, and delivery of production systems becomes essential. The majority of organizations rely on tools such as Kubernetes [1] or HashiCorp Nomad [2]; however, these platforms often impose high operational costs and require substantial technical expertise to maintain. For example, deploying a Kubernetes cluster requires at least 4 CPU cores and 16 gigabytes of RAM for the control-plane component alone, and even then, managing upgrades, configuring networking and storage, handling security policies, and troubleshooting failures remain highly complex tasks that often demand dedicated DevOps teams and extensive ongoing maintenance.

The *orchestration principle* [3], [4] underlies the aforementioned platforms. Recent advances in self-healing systems [5] and zero-trust security models [6] provide additional mechanisms for enhancing system reliability and security. Orchestration refers to the automated management, coordination, and deployment of

multiple services and components in distributed systems to ensure their consistent and efficient operation. However, modern orchestration platforms are often difficult to modify and extend. For instance, platforms that rely on a push-based scheduling model are susceptible to issues such as delivery failure following network connectivity errors and workload duplication. For businesses, these issues can be critical, since infrastructure failures may result in significant financial losses.

The complexity of existing orchestration platforms, combined with the challenges of maintainability and workload management, creates a significant barrier for organizations that cannot afford large DevOps departments. Reliance on a push-based scheduling model limits fault tolerance and introduces bottlenecks in distributed coordination. This thesis addresses these limitations by exploring a pull-based approach to distributed workload management. In the pull-based model, rather than directing a worker node to perform a specific task, the task is placed in a shared registry from which worker nodes independently retrieve and execute it. In practice, instead of receiving instructions from a central controller, worker nodes periodically poll the control plane for available workloads and autonomously claim ownership through a lease mechanism. This model aims to increase reliability, simplify maintenance, and make distributed orchestration more accessible.

The purpose of this work is to design and implement a working prototype of a distributed workload management system based on a *pull-model approach*. The key objectives are as follows: reducing operational overhead; designing a decentralized architecture with the ability to scale; improving security by automating manual infrastructure operations; and creating a lightweight, flexible alternative to existing orchestration platforms. During each phase, artifacts will be collected

to ensure the solution can be replicated.

To achieve the stated objectives, the Design Science Research [7] methodology is applied, which focuses on building and evaluating artifacts to solve real-world problems. This approach emphasizes iterative development, where each stage informs and refines the next, ensuring that the resulting system effectively addresses the identified challenges. The work is organized into three stages:

1. Problem identification and requirements clarification: analyzing existing orchestration solutions, identifying their limitations, and defining the functional and non-functional requirements for the proposed system.

2. System design and prototype development: developing the architecture and implementing a working prototype of the distributed workload management system, incorporating features such as task scheduling, fault tolerance, and scalability.

3. Validation and evaluation: testing the prototype under controlled and realistic conditions, measuring its performance, reliability, and resource efficiency, and comparing it against existing platforms to assess improvements and identify areas for further refinement.

This methodology ensures a structured, evidence-based approach to designing and assessing the proposed system while keeping practical applicability as a central focus.

Generative artificial intelligence (GenAI) tools were used in this research as a language and editing aid. GenAI assisted in refining phrasing, improving grammatical correctness, and enhancing the academic tone of the exposition.

The expected outcome of this research is a lightweight and reliable orchestration system prototype that demonstrates the practical benefits of applying a pull-based model in distributed workload management. The developed system is expected to reduce operational overhead by automating configuration delivery and minimizing manual interventions. It will introduce a decentralized control plane that relies on REST API and PostgreSQL for efficient coordination and data storage. Additionally, the work aims to provide a clear comparison with existing orchestration platforms, highlighting improvements in scalability, fault tolerance, and resource efficiency. As a result, this thesis contributes to simplifying infrastructure management and offers an alternative orchestration approach that can be more easily adopted by organizations with different operational scales.

1.1 Research Questions

This thesis seeks to answer the following research questions:

RQ1. How can a pull-based orchestration approach reduce operational complexity and resource overhead compared to traditional push-based platforms like Kubernetes, while maintaining comparable fault tolerance and self-healing capabilities?

RQ2. What are the architectural trade-offs inherent in a fully pull-based orchestration model, particularly with respect to consistency guarantees, coordination overhead, fault isolation mechanisms, and the trade-off between scheduling simplicity and placement optimality?

1.2 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 1: Introduction Motivates the problem, states the research questions, outlines the Design Science Research methodology, and provides an overview of the thesis structure.

Chapter 2: Problem Identification Analyzes the business impact of orchestration failures, examines existing solutions (Kubernetes, HashiCorp Nomad), and derives functional and non-functional requirements for the proposed system.

Chapter 3: Literature Review Investigate modern container orchestration systems and decentralized, pull-based approaches in distributed computing, highlighting gaps in current research and practice.

Chapter 4: Implementation Presents the system architecture, design decisions, and technical implementation details through key scenarios, demonstrating how the pull-based model operates in practice.

Chapter 5: Evaluation and Discussion Evaluates the prototype against the established requirements, measures performance metrics (including RTO achievement), and discusses the architectural trade-offs observed in the pull-based model.

Chapter 6: Conclusion Summarizes the contributions, limitations, and practical implications of the research, and outlines directions for future work.

Chapter 2

Literature Review

Several literature sources were employed to perform analysis of the problem. Google Scholar and Research Gate served to perform the research on the distributed compute management and pull-model approach. The keywords used to find the sources were "cloud orchestration", "distributed cloud computing", "pull-model". To fetch more practical insights we decided to use official Kubernetes, HashiCorp Nomad and Apache OpenServerless documentation.

2.1 Container orchestration systems

Modern orchestration platforms, such as Kubernetes and HashiCorp Nomad, have become standard tools for managing distributed workloads in cloud environments. Kubernetes provides a declarative configuration model and uses a centralized control plane to manage containerized workloads across multiple nodes [1]. However, it relies on hybrid (both push and pull models) control mechanism. To decide where to schedule workloads, there is a push-based mechanism, located on the control plane, which could serve as another point of failure. But

for configuration delivery it uses pull model, node agent watches for changes and applies it when there are some. Comprehensive surveys of Kubernetes scheduling [8], [9] classify its scheduling strategies into filter-score-prioritize pipelines and identify open challenges in multi-cluster and edge deployments. Nomad offers a similar architecture but emphasizes simplicity and flexibility in scheduling non-containerized workloads [2]. Nevertheless agent or client in terms of Nomad acts like a reporter of a node current state whether a resource operator. In conclusion both rely primarily on a push-based control mechanism, where the control plane assigns tasks to worker nodes. While this approach ensures strong control and consistency, it introduces higher coupling, operational complexity, and reduced fault tolerance.

Similarly, research on service mesh architectures [10] demonstrates that eliminating sidecar proxies through novel control plane designs can significantly reduce control plane overhead compared to traditional approaches like Istio. This sidecar-free architecture for multi-tenant environments provides a viable alternative to heavily-layered orchestration systems, aligning with the lightweight design principles explored in this thesis.

Research by Sebrechts et al. [3] highlights challenges in distributed application management, emphasizing the need for scalable and adaptive orchestration systems. It offers great alternative - distributing operational agents. Similarly, Pantazoglou et al. [11] propose decentralized workload management approaches to improve energy efficiency and reduce coordination overhead in large-scale systems. These studies suggest that centralization, while powerful, can become a bottleneck for resilience and scalability.

The evolution of cluster scheduling architectures further illustrates this tension. Google's Borg [12] employs a centralized monolithic scheduler with per-

machine agents (Borglets), while its successor Omega [13] introduced shared-state optimistic concurrency to improve scalability. Apache Mesos [14] and YARN [15] adopted two-level scheduling, decoupling resource management from application-specific scheduling logic. More recently, Sparrow [16] demonstrated that stateless, distributed scheduling via random sampling can achieve low-latency placement for short-lived tasks, and Apollo [17] explored coordinated decentralized scheduling with per-job managers. Hawk [18] proposed a hybrid approach, delegating short tasks to distributed schedulers while retaining centralized control for long-running jobs. Firmament [19] applied min-cost max-flow optimization to centralized cluster scheduling. These systems collectively demonstrate a progression from monolithic to shared-state to fully distributed designs, yet all retain some form of centralized coordination that the pull-based model proposed in this thesis seeks to minimize.

2.2 Pull-based and decentralized approaches

The pull-based paradigm, originally explored in software engineering and version control systems, introduces an alternative model for coordination and synchronization. Recent work on pull-based scheduling for serverless computing [20] demonstrates that eliminating centralized scheduling decisions in favor of worker-initiated workload acquisition can reduce coordination overhead and improve scalability. These principles can be adapted to workload orchestration, where worker nodes autonomously retrieve workloads and configurations, thus minimizing reliance on a central scheduler.

Recent research on decentralized cloud computing supports this shift. Pantazoglou et al. [11] propose decentralized and energy-efficient workload manage-

ment approaches that reduce coordination overhead in enterprise clouds, while Hiku [20] demonstrates the viability of pull-based scheduling in serverless environments. These approaches align with modern distributed architectures that prioritize resilience and fault isolation over strict central coordination.

The CAP theorem [21], [22] formalizes the fundamental trade-off between consistency, availability, and partition tolerance in distributed systems. Brewer [21] argues that modern systems can mitigate CAP limitations through techniques such as eventual consistency and conflict-free replicated data types, which aligns with the design philosophy of the proposed system. Amazon's Dynamo [23] exemplifies an availability-first, eventually consistent design that inspired many cloud-native architectures. In contrast, traditional coordination services such as ZooKeeper [24] and Google's Chubby [25] provide strong consistency through consensus-based distributed locking, while Paxos [26] and Raft [27] offer theoretically grounded consensus protocols at the cost of increased coordination overhead. The pull-based orchestration model proposed in this thesis delegates coordination to a relational database rather than implementing application-level consensus: by delegating scheduling decisions to autonomous agents that poll for available workloads, the system favors availability and partition tolerance over strong consistency, a trade-off justified by the workload characteristics and operational requirements identified in the problem analysis.

Comprehensive surveys of geo-distributed task scheduling [28] further emphasize that distributed systems operating across heterogeneous network conditions and region-specific resource constraints require sophisticated coordination approaches. Such systems must balance performance enhancement, fairness assurance, and fault tolerance improvement while managing challenges inherent to geographic distribution, including varying network latency and computational

capabilities across locations.

2.3 Security and reliability concerns

Infrastructure reliability and security are critical for large-scale deployments. Google Cloud's production systems rely on strong security models and automated workload verification to ensure service integrity [29]. However, such designs also depend on highly centralized control planes, which can create single points of failure. Research on resilient cloud systems emphasizes the need for autonomous components that can recover independently [30] and sustain partial system failures [31], [32]. Finally, compliance and governance are integral to reliable infrastructure, especially in enterprise and regulated environments. Emerging research also highlights the potential of decentralized and federated cloud management frameworks to enhance resilience and security while reducing reliance on single points of failure.

2.4 Lightweight and edge orchestration

The demand for lightweight orchestration has intensified with the proliferation of edge computing and resource-constrained IoT environments. Čilić et al. [33] benchmark Kubernetes, K3s, and KubeEdge on edge hardware, demonstrating that even lightweight Kubernetes distributions incur significant overhead relative to bare-metal execution. Usman et al. [34] extend this analysis to lightweight distributions including K3s and k0s, finding that resource consumption remains substantial for small-scale edge deployments. Böhm and Wirtz [35] survey Kubernetes-based orchestration architectures for smart city edge scenar-

ios and identify architectural complexity as a barrier to adoption in resource-constrained settings.

Alternative frameworks have emerged to address these limitations. FogBus2 [36] provides a lightweight container-based framework integrating IoT, edge, and cloud resources with minimal operational overhead. Arora et al. [37] propose a lightweight edge slice orchestration framework for multi-access edge computing with a minimal control plane. Gand et al. [38] introduce a self-adaptive fuzzy controller for lightweight edge container orchestration, demonstrating that autonomous reconciliation mechanisms can maintain service quality without centralized scheduling. Wang et al. [39] examine container orchestration in edge and fog environments for real-time IoT applications, emphasizing the need for low-overhead scheduling mechanisms. Morabito et al. [40] explore consensus-based distributed orchestration for edge microservices, though their approach introduces coordination overhead that the pull-based model seeks to avoid.

2.5 Fault tolerance in container-based systems

Fault tolerance mechanisms in container orchestration span failure detection, state recovery, and workload rescheduling. Bakhshi and Rodriguez-Navas [41] study fault-tolerant permanent storage for container-based fog architectures, highlighting that stateless compute layers combined with durable storage backends provide an effective resilience pattern. The seminal work on leases by Gray and Cheriton [42] establishes time-based distributed locking as a lightweight alternative to consensus-based coordination, a principle directly applicable to workload claiming in pull-based systems. MapReduce [43] introduced the pattern of automatic task re-execution upon failure, inspiring the reconciliation-based

fault tolerance mechanisms adopted in modern orchestrators. TetriSched [44] addresses adaptive rescheduling in shared heterogeneous clusters, demonstrating that dynamic plan revision can improve placement quality, albeit at the cost of increased scheduling complexity.

2.6 Summary

The reviewed literature provides valuable information on modern approaches to workload management, orchestration and security in cloud environments. Existing systems such as Kubernetes and Nomad demonstrate effective solutions but still rely heavily on centralized management models rooted in decades of cluster scheduling research [12]–[15]. Research shows that decentralized and pull-based mechanisms can increase scalability, resilience and autonomy. CAP theory [21], [22] and the design of availability-first systems [23] provide theoretical justification for architectures that favor partition tolerance over strong consistency. Studies on lightweight edge orchestration [33], [34], [36], [37] demonstrate demand for minimal-footprint scheduling solutions. Overall, while much can be gained from the work already done, there is still considerable room for improvement - especially in developing more adaptive, fault-tolerant and secure orchestration architectures.

Chapter 3

Problem identification

3.1 Glossary

Recovery Time Objective (RTO) – a formal metric defined within business continuity and disaster recovery planning that specifies the maximum acceptable duration a system, application, or service may remain unavailable after a disruption before significant business impact occurs. RTO directly influences architectural decisions regarding redundancy, failover strategies, and automation levels; a lower RTO typically necessitates more sophisticated and costly resilience mechanisms.

Requests Per Second (RPS) – a performance indicator quantifying the number of incoming service requests a system processes in one second. RPS serves as a key benchmark for evaluating throughput, scalability, and load handling capacity, particularly in web-based and API driven applications. It is often used in conjunction with latency and error rate metrics to assess overall system efficacy under load.

Service-Level Agreement (SLA) – a contractually binding commitment

between a service provider and a client that defines the expected level of service, including measurable performance criteria such as uptime percentage, response time, RTO, and support responsiveness. Violations of SLA terms may incur financial penalties or service credits, making SLA adherence a critical driver of system design and operational practices in commercial and enterprise contexts.

3.2 Business impact

To illustrate the practical significance of workload orchestration in real-world systems, consider a business scenario involving a B2B authentication-as-a-service provider. Such a company operates an API service responsible for authenticating incoming client requests and determining access permissions. Upon initial deployment, the service handles a baseline load of 10 RPS. In a conventional deployment model, the API is hosted on a single virtual machine (VM). However, this approach introduces a critical single point of failure: if the underlying physical server hosting the VM fails, the entire service becomes unavailable, resulting in service downtime.

In the example described, the failure goes unnoticed for approximately one minute. Engineering staff then manually migrate the service to another VM, resulting in a RTO of five minutes. During this outage window, the system fails to process 3,000 authentication requests (calculated as 10 RPS $\times$ 300 seconds). Such service interruptions inevitably lead to both reputational damage and direct financial losses, particularly in B2B contexts where SLA often stipulate strict uptime guarantees and penalties for non-compliance.

In contrast, had the service been deployed within an orchestrated infrastructure such as one managed by Kubernetes or another container orchestration

platform the failure scenario would unfold differently. Upon the VM's failure, the orchestration system would automatically detect the loss of the service instance and reschedule it onto a healthy node within the cluster. Assuming a detection latency of one minute and a recovery time of 30 seconds, the total RTO would be reduced to 90 seconds. Consequently, only 900 requests (10 RPS × 90 seconds) would be lost a 70% reduction in missed requests compared to the non-orchestrated scenario.

This example underscores the critical role of orchestration in enhancing system resilience, minimizing recovery times, and maintaining service continuity. In contemporary distributed systems, where availability and reliability are paramount, orchestration is not merely a convenience but a foundational requirement for operational robustness and business sustainability.

3.3 Analysis of the existing solutions

3.3.1 Kubernetes

Kubernetes - also known as K8s, is an open source system for automating deployment, scaling, and management of containerized applications [1].

K8s has become the de facto standard for container orchestration in modern cloud-native environments. Originally inspired by Google's internal Borg system [12] a large-scale cluster management platform developed over a decade of operational experience Kubernetes was open-sourced by Google in 2014 and subsequently donated to the Cloud Native Computing Foundation (CNCF) in 2015 [45]. This strategic move catalyzed widespread community adoption, ensured vendor neutrality, and established a sustainable governance model that continues

to drive its evolution. As of early 2026, the official Kubernetes GitHub repository[1] boasts over 120,000 stars and contributions from more than 4,000 individual developers, reflecting its robust ecosystem and strong industry endorsement.

Today, Kubernetes extends far beyond basic workload orchestration. It provides a comprehensive platform for automating deployment, scaling, service discovery, load balancing, secrets management, rolling updates, and self-healing effectively serving as a foundational layer for microservices architectures, serverless frameworks, and GitOps-based CI/CD pipelines [46]. Its extensibility via Custom Resource Definitions (CRDs) and operators enables domain-specific automation, further cementing its role as an infrastructure control plane.

However, despite its extensive capabilities and broad adoption, Kubernetes introduces notable operational and economic challenges that warrant careful consideration particularly for small-to-medium enterprises or resource-constrained environments. Two primary concerns are examined below.

Infrastructure Cost and Resource Overhead

Running a production-grade, highly available (HA) Kubernetes cluster entails significant infrastructure overhead. The control plane components such as the API server, scheduler, and controller manager must be replicated for fault tolerance, while the distributed key-value store etcd (which maintains cluster state) requires its own dedicated and co-located resources to ensure consistency and low-latency access.

A minimally viable HA configuration typically demands:

- **Three** virtual machines (VMs) with at least 2 vCPUs and 2 GB RAM each

[1] https://github.com/kubernetes/kubernetes

for the Kubernetes control plane;

- **Three** additional VMs (often collocated with the control plane but ideally isolated for performance and security) with 2 vCPUs and 4 GB RAM each to host an HA etcd cluster;
- **At least one** worker node with 1 vCPU and 512 MB RAM to schedule application workloads.

This baseline setup consumes approximately 13 vCPUs and 18.5 GB RAM of provisioned resources, yet yields only 1 vCPU and 512 MB RAM of net usable capacity for actual business applications a resource efficiency ratio of less than 8%. While lightweight Kubernetes distributions such as K3s [47] and K0s [48] aim to reduce this footprint by consolidating components and trimming non-essential features, they often sacrifice advanced capabilities (e.g. full RBAC granularity, audit logging, or native support for certain CSI drivers) or introduce trade-offs in security and observability. Evaluations of lightweight Kubernetes distributions on edge hardware [33], [34] confirm that resource consumption remains significant even with stripped-down deployments. Consequently, organizations must weigh the benefits of Kubernetes feature richness against its operational footprint, especially when workloads are modest or latency-sensitive.

Operational Complexity and Expertise Requirements

Kubernetes is renowned for its steep learning curve and inherent architectural complexity. Its declarative model, while powerful, requires a deep understanding of concepts such as pods, services, ingress controllers, network policies, persistent volumes, and reconciliation loops. Misconfigurations particularly in

networking, resource quotas, or security contexts are common and can lead to outages, performance degradation, or security vulnerabilities.

Maintaining a production Kubernetes cluster reliably demands specialized expertise. Industry surveys indicate that organizations typically employ at least one full-time Site Reliability Engineer (SRE) or platform engineer per Kubernetes cluster in mission-critical environments. These professionals must not only manage day-to-day operations but also implement monitoring (e.g. via Prometheus and Grafana), logging (e.g. EFK or Loki stacks), backup/restore strategies, and compliance controls. The scarcity of qualified Kubernetes specialists significantly elevates operational expenditures, particularly in the form of personnel costs. In the Russian market, Site Reliability Engineers (SREs) with proficiency in Kubernetes command monthly salaries ranging from approximately 240,000 to 280,000 RUB (net), with the national average reported at 229,000 RUB in 2025 [49]. These figures reflect a highly competitive labor market, especially in metropolitan hubs such as Moscow, where compensation approaches the upper end of this range. Internationally, the cost burden is even more pronounced: in North America and Western Europe, average annual SRE salaries exceed $130,000–$160,000 as of 2025 [50]. Consequently, maintaining an in-house Kubernetes platform often necessitates substantial investment not only in infrastructure but also in specialized human capital a barrier that disproportionately affects smaller organizations and startups with constrained budgets.

This complexity has spurred the growth of managed Kubernetes offerings (e.g. Amazon EKS, Google GKE, Azure AKS), which offload control plane management to cloud providers. While these services reduce operational burden, they introduce vendor lock-in risks and often shift rather than eliminate costs to the infrastructure and networking layers.

3.3.2 HashiCorp Nomad

HashiCorp Nomad is an open-source workload orchestrator designed to simplify the deployment and management of containerized, virtualized, and batch applications across distributed infrastructure [2]. Originally released by HashiCorp in 2015, Nomad positions itself as a simpler, more lightweight alternative to Kubernetes, emphasizing ease of operation and flexibility in workload types. While it lacks the overwhelming market dominance of Kubernetes, Nomad has established a dedicated user base particularly among organizations seeking a less complex orchestration solution. As of early 2026, the official Nomad GitHub repository[2] reflects a mature though smaller-scale project within the HashiCorp ecosystem.

Today, Nomad provides a versatile platform for orchestrating diverse workloads including Docker containers, raw executables, Java applications, and QEMU-based virtual machines. Its architecture centers on a single binary that handles both scheduling and execution, eliminating the need for external coordination services such as etcd. Nomad integrates natively with other HashiCorp tools most notably Consul for service discovery and Vault for secrets management forming a cohesive infrastructure automation stack [2]. This integration, combined with support for multi-region federation and heterogeneous workloads, has made Nomad particularly attractive for edge computing scenarios and hybrid cloud environments where simplicity and resource efficiency are paramount.

However, despite its architectural simplicity and operational advantages, Nomad presents certain limitations and concerns that warrant careful evaluation. Two primary areas are examined below: infrastructure overhead and operational complexity relative to the proposed pull-based approach.

[2] https://github.com/hashicorp/nomad

Infrastructure Cost and Resource Overhead

Nomad is explicitly designed to minimize infrastructure footprint, and in this regard it succeeds significantly better than Kubernetes. The control plane consists of a single binary approximately 80-100 MB in size that combines the API server, scheduler, and state management. A minimal high-availability Nomad cluster typically requires:

- **Three** server nodes with 1 vCPU and 512 MB RAM each to form a Raft [27] quorum for state consensus;
- **No additional infrastructure** for coordination storage, as state is embedded within the server binary via Raft;
- **Client nodes** with base memory overhead of approximately 50-100 MB per agent, compared to Kubernetes kubelet requirements of several hundred megabytes.

This baseline configuration consumes roughly 3 vCPUs and 1.5-2.25 GB RAM for the control plane, representing a resource efficiency ratio substantially superior to Kubernetes. Real-world deployments report that Nomad-based clusters can reduce server costs by approximately 20% compared to equivalent Kubernetes setups [2]. In resource-constrained environments such as edge devices or IoT gateways, this difference becomes critical.

Operational Complexity and Architectural Concerns

Nomad's primary value proposition lies in its operational simplicity relative to Kubernetes. The learning curve for basic Nomad operations typically requires

approximately two weeks, compared to two months or more for production-grade Kubernetes proficiency. Configuration uses HashiCorp Configuration Language (HCL) which is generally more concise than Kubernetes YAML. However, Nomad retains a fundamentally push-based scheduling model: the central control plane evaluates job submissions and proactively pushes allocations to client nodes, rather than having agents pull work on-demand [2]. While this approach is simpler than Kubernetes multi-component architecture, it still requires the control plane to maintain cluster-wide state and make centralized scheduling decisions, contrasting with the fully decentralized pull model explored in this thesis.

Additionally, Nomad faces significant ecosystem and licensing challenges. The community and third-party tooling around Nomad remain substantially smaller than Kubernetes ecosystem. Fewer monitoring integrations, CI/CD plugins, and commercial support options exist for Nomad. More critically, in August 2023 HashiCorp transitioned Nomad from the Mozilla Public License (MPL) 2.0 to the Business Source License (BSL) 1.1 a move that sparked controversy and raised concerns about vendor lock-in. Unlike Kubernetes which operates under the permissive Apache 2.0 license and is governed by the vendor-neutral Cloud Native Computing Foundation, Nomad is now owned by IBM following its $6.4 billion acquisition of HashiCorp in 2024. This combination of a restrictive source-available license and corporate ownership introduces long-term adoption risks for organizations requiring genuinely open-source infrastructure components with sustainable community governance.

3.4 Requirements clarification

Following a comprehensive competitive analysis and a thorough understanding of business imperatives, the next phase involves the systematic elicitation and formalization of requirements for the proposed orchestration prototype. To ensure alignment between stakeholder expectations and technical implementation, requirements are categorized the following way: business scenarios (representing business and operational perspectives), functional requirements (defining system capabilities), and non-functional requirements (specifying quality attributes and constraints).

3.4.1 Business Scenarios

The business scenarios were represented as user stories. No stakeholder interviews were conducted for this phase; instead, decisions were based on prior analysis. The following user stories were derived from that analysis and capture needs across business leadership, technical management, and operational engineering roles. These narratives were identified as central to the system's value proposition.

- *As a business owner, I want IT services to experience minimal or no downtime so that customer trust and revenue streams remain uninterrupted.*
- *As a Chief Technology Officer (CTO), I want infrastructure failures not to result in significant operational or financial losses.*
- *As a Site Reliability Engineer (SRE), I want the failure of a base infrastructure component such as a virtual machine to trigger automatic remediation without manual intervention.*

- *As a SRE, I want application workloads to be automatically rescheduled onto healthy, available nodes upon VM failure.*
- *As a SRE, I want a unified ingress point (API gateway) for all externally exposed applications. For the prototype scope, supported protocols shall be HTTP-based: RESTful APIs, gRPC over HTTP/2, and GraphQL.*

3.4.2 Functional Requirements

Derived directly from the above user stories, the following functional requirements specify the system's mandatory behaviors:

FR1. The system shall be distributed and fault-tolerant, ensuring continued operation despite the failure of individual compute nodes.

FR2. The system shall automatically detect node (e.g., VM) failures and reschedule affected workloads onto healthy nodes within the cluster.

FR3. The system shall provide a unified ingress layer (API gateway) capable of routing external traffic to internal services based on protocol and path, with initial support limited to HTTP-based protocols: REST, gRPC (via HTTP/2), and GraphQL.

3.4.3 Non-Functional Requirements

To ensure the system meets real-world operational standards, the following non-functional requirements critical for reliability, security, and maintainability were established:

NFR1. **Recovery Time**: Workload rescheduling following a node failure must complete within **90 seconds** (inclusive of failure detection and service restoration).

NFR2. **Control Plane Resilience**: Running workloads shall remain operational during transient control plane outages. Because agents manage containers locally and the control plane holds no runtime state, a control plane failure does not affect already-running workloads. New scheduling and failure recovery require a functioning control plane, which can be made highly available through replication behind a load balancer.

NFR3. **Scalability**: The architecture shall support horizontal scaling of both control and data planes to accommodate future growth in workload volume and cluster size without architectural refactoring.

NFR4. **Security**: All inter-component communication shall be encrypted in transit using Transport Layer Security (TLS) 1.3 or higher. Mutual TLS (mTLS) shall be enforced for service-to-service communication within the cluster to ensure authentication and data integrity.

3.5 Requirements Finalization

Taken together, these requirements reflect a deliberate balance between operational pragmatism, business continuity, and technical feasibility. They establish a clear design envelope for the prototype: a lightweight, self-healing orchestration layer that prioritizes rapid recovery, protocol-aware ingress, and secure communication while explicitly avoiding the resource overhead and administrative complexity associated with full-scale Kubernetes deployments. The 90-second

RTO serves as a measurable benchmark for validation, and the emphasis on control plane resilience addresses a key gap in existing lightweight orchestration solutions. Ultimately, this requirements baseline not only encodes stakeholder expectations but also provides a testable foundation against which architectural alternatives and implementation outcomes can be rigorously evaluated.

Chapter 4

Implementation

4.1 System Design

4.1.1 Pull vs. Push Models

A fundamental architectural decision in distributed orchestration systems concerns the mechanism by which configuration updates and workload instructions are propagated from the control plane to worker nodes. Two predominant paradigms exist: the Push model and the Pull model. These differ not only in data flow direction but also in their underlying philosophy imperative execution versus declarative reconciliation with significant implications for scalability, resilience, and operational complexity.

The Push model operates imperatively: a central controller initiates synchronous, outbound connections to target nodes to enforce configuration changes immediately upon request. Tools such as Ansible exemplify this approach, often leveraging secure shell (SSH) or Windows Remote Management (WinRM) without requiring persistent agents.

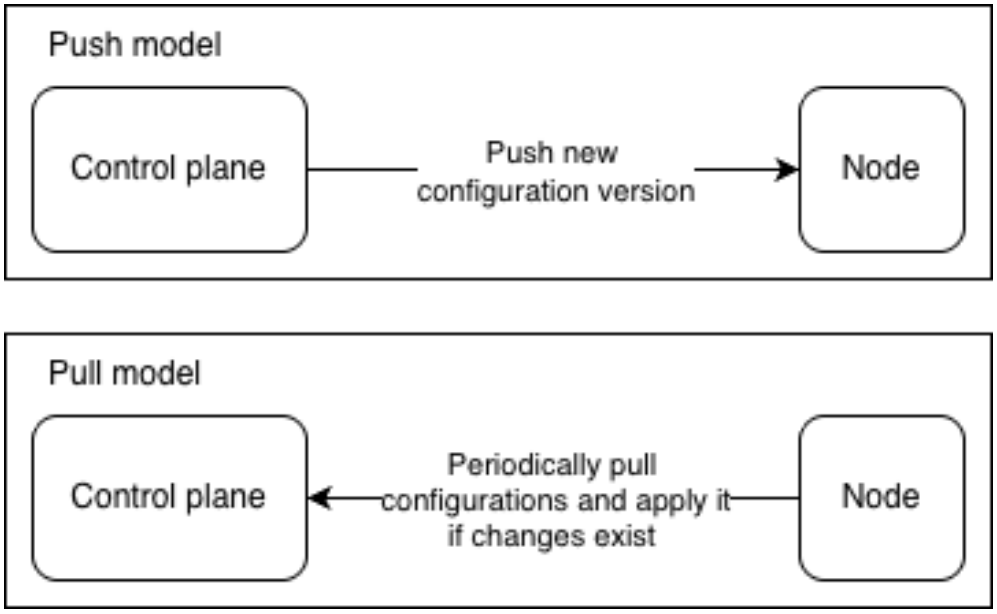

Fig. 4.1. Comparison of Push and Pull configuration propagation models. In the Push model (top), the control plane initiates outbound connections to apply changes directly to nodes. In the Pull model (bottom), each node periodically polls the control plane or a configuration repository to reconcile its local state with the desired state.

Key advantages include:

- **Immediate enforcement**: changes are applied with minimal delay once triggered by an operator.
- **Predictable timing**: the operator retains full control over when a change is deployed.
- **Simplified auditing**: all modifications originate from a single, identifiable source.
- **Reduced node-side footprint**: agentless implementations (e.g., Ansible) avoid long-running processes on worker nodes.

However, this model presents notable limitations in large-scale or dynamic environments:

- **Scalability bottlenecks**: the central controller becomes a performance bottleneck when managing thousands of nodes concurrently.
- **Network constraints**: requires bidirectional connectivity from the controller to every node problematic in environments with firewalls, NAT, or private subnets.
- **State drift vulnerability**: nodes that experience local configuration changes or reboots remain out of sync until the next manual or scheduled push.
- **Expanded attack surface**: the controller must securely authenticate to all nodes, often necessitating complex credential or key management.

In contrast, the Pull model is inherently declarative: each node runs a lightweight agent that periodically (e.g., every 10 seconds) queries a central configuration source such as an API server or Git repository and autonomously reconciles its local state with the declared desired state.

This approach offers several critical benefits aligned with modern reliability engineering principles:

- **Automatic recovery**: nodes automatically recover from configuration drift or transient failures without external intervention.
- **Horizontal scalability**: load is distributed across nodes; the control plane serves read requests rather than initiating writes.
- **Network resilience**: nodes only require outbound connectivity (e.g., HTTPS to a config server), making the model compatible with restrictive network topologies.

- **Decoupled architecture**: failures in the control plane do not immediately halt node operations; nodes continue functioning with their last known good state.

Nonetheless, the Pull model introduces its own trade-offs:

- **Enforcement latency**: configuration changes are applied only at the next polling interval, unless supplemented with event-driven triggers (e.g., webhooks).
- **Agent overhead**: requires a persistent process on each node, increasing resource consumption and attack surface at the edge.
- **Temporal inconsistency**: nodes may operate with slightly divergent configurations until synchronization occurs.
- **Observability complexity**: determining the exact time of state convergence across the fleet requires additional telemetry.

Upon thorough analysis, it becomes evident that the requirements established in Section 3.4 particularly the 90-second Recovery Time Objective (RTO), resilience to infrastructure failures, and support for autonomous recovery align closely with the core strengths of the Pull model. The capacity for automatic recovery, network resilience, and decentralized operation directly addresses the business imperatives of minimizing downtime and reducing manual intervention.

Consequently, the Pull model has been selected as the foundational architectural paradigm for the proposed prototype. This choice is consistent with the broader trend toward availability-first distributed system design [21], [23], where

partition tolerance and autonomous operation are prioritized over strong consistency. Furthermore, the design will incorporate mitigations to address its inherent limitations:

A configurable, short polling interval to bound recovery latency; Optional event-based push notifications to trigger immediate reconciliation when critical updates occur; Minimalist agent design to reduce resource footprint; Centralized telemetry to monitor configuration convergence across the cluster. This hybrid-aware Pull approach ensures adherence to non-functional requirements while preserving operational simplicity and scalability.

4.1.2 Top-Layer Architecture

Given the explicit non-functional requirements for decentralization, fault tolerance, and automated recovery, the system must be designed from the ground up as a distributed architecture. Modern infrastructure platforms typically decompose into two orthogonal domains: the control plane and the data plane. This separation enables scalability, operational clarity, and independent evolution of concerns.

The control plane serves as the authoritative source of truth for the desired system state. It maintains metadata about workloads (e.g., placement rules, resource constraints, health policies) and exposes interfaces for human (e.g., SRE) and programmatic interaction. Critically, it does not execute user workloads but orchestrates their placement and lifecycle.

The data plane consists of a fleet of homogeneous infrastructure units - virtual machines, that host and execute actual workloads. These units are treated as cattle, not pets: individually ephemeral and replaceable, but collectively resilient [51].

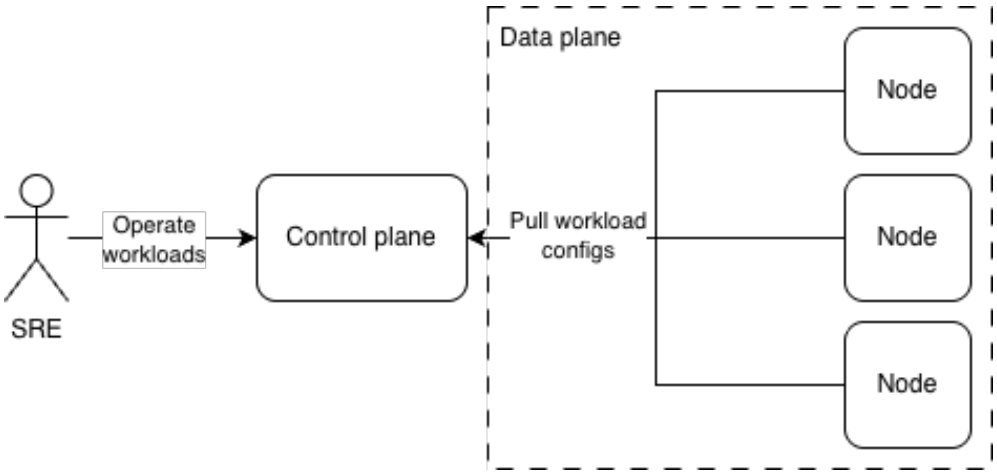

Fig. 4.2. High-level component diagram of the proposed system. The control plane (central registry) communicates with data-plane nodes via a REST API. Each node runs a local agent that periodically pulls configuration state. An SRE interacts with the control plane to declare desired workload placements.

This clean separation ensures that failures in the data plane do not compromise global coordination, while control-plane resilience mechanisms (discussed below) prevent single points of failure.

4.1.3 Control Plane

The control plane is a mission-critical component: without it, new workloads cannot be scheduled, and failed workloads cannot be rescheduled. To meet the requirement of resilience to control-plane downtime (NFR2), the control plane must be designed as stateless in its compute layer, enabling horizontal scaling and active-active replication across multiple instances.

Persistent Storage

A durable, strongly consistent datastore is required to persist the desired state of the system. After evaluation of alternatives (including etcd, Consul, and

embedded SQLite), PostgreSQL was selected as the primary storage backend due to the following advantages: open-source and permissively licensed, ensuring cost efficiency and freedom from vendor lock-in; mature, actively maintained, with a large community and enterprise-grade tooling; ACID-compliant, providing strong consistency guarantees essential for orchestration correctness; widely adopted in production systems, simplifying operational expertise and troubleshooting.

Given the expected workload primarily metadata operations (create, read, update of service definitions) PostgreSQL is more than sufficient in terms of throughput and latency. If scaling limits be approached (e.g., >10k operations/second), the architecture supports migration to PostgreSQL-compatible distributed databases such as YugabyteDB or CockroachDB with minimal application-level changes.

Application Interface

To enable communication between the control plane and data-plane agents, a well-defined RESTful API is exposed over HTTPS. While alternatives like gRPC offer superior performance and streaming capabilities, REST was chosen for the following reasons: simplicity of implementation and debugging during the prototype phase; broad tooling support (curl, Postman, standard HTTP clients); sufficient adequacy for the Pull model, where agents perform periodic, low-frequency polling (e.g., every 30 seconds); ease of integration with future API gateway extensions (e.g., authentication, rate limiting).

All techincal API endpoints enforce mutual TLS (mTLS) to ensure confidentiality and authenticity of control-plane communications.

Ingress

Since workloads may be scheduled on any node within the data plane, services that require external exposure via HTTP endpoints necessitate a unified ingress point to ensure consistent and discoverable access. To fulfill this requirement, a new dedicated component (henceforth referred to as the Edge) is introduced. The Edge component continuously discovers active workloads that declare HTTP-based network exposure (e.g., via annotations or configuration metadata) and dynamically configures routing rules to proxy incoming requests to the appropriate backend instances over a single, canonical endpoint.

The Edge operates as a lightweight, programmable reverse proxy integrated into the orchestration control plane. While production-grade alternatives such as NGINX or Envoy could serve as drop-in replacements for the Edge component, their adoption would require additional engineering effort to implement real-time synchronization of downstream service endpoints typically achieved through integration with service discovery mechanisms or dynamic configuration APIs (e.g., Envoy's xDS protocol). For the scope of this prototype, a minimal custom implementation is preferred to maintain architectural simplicity and reduce external dependencies, while retaining the flexibility to migrate to standardized proxy solutions in future iterations.

4.1.4 Data Plane

The data plane comprises the physical or virtual infrastructure units that execute workloads. Consistent with cloud-native principles, these units are homogeneous, ephemeral, and automatically managed no single node is considered reliable, but the collective ensemble delivers high availability.

Agent

Each node in the data plane runs a lightweight agent process, whose responsibilities include:

- Periodically polling the control plane's REST API to retrieve the latest desired configuration
- Comparing the desired state with the current local state
- Applying necessary changes (e.g., starting/stopping containers, updating network rules)
- Reporting node health and workload status back to the control plane

The agent embodies the Pull model and enables automatic recovery: if a node reboots or is replaced, the agent automatically re-synchronizes with the control plane, ensuring eventual consistency. This reconciliation pattern follows the principle of declarative state convergence established in large-scale distributed systems [43].

Workload Execution

Workloads are executed in containers to guarantee portability, isolation, and reproducibility across environments. The system uses the Docker Engine as its default container runtime due to its ubiquity, rich ecosystem, and mature tooling. However, the architecture is runtime-agnostic: the agent interfaces with the container runtime via a pluggable abstraction layer (e.g., using the Docker-compatible HTTP API). This design permits future migration to alternative runtimes such as

Podman (rootless, daemonless) or CRI-O (Kubernetes-optimized) without modifying core orchestration logic.

Container images are pulled from a secure, authenticated registry, and all runtime configurations (CPU, memory, network) are derived from the declarative specifications stored in the control plane.

This layered, decoupled architecture directly addresses the functional and non-functional requirements:

- Fault tolerance via agent-driven automatic recovery (NFR1)
- Control-plane resilience through stateless design and PostgreSQL durability
- Scalability via homogeneous data-plane units and REST-based coordination
- Security via mTLS and container isolation

4.1.5 Architecture Finalization

Having examined each constituent component of the proposed system the control plane, data plane, agent, Edge ingress layer, and their interconnections it is now possible to consolidate these elements into a coherent and implementable architectural blueprint. The resulting design demonstrates strong alignment with both the functional and non-functional requirements established in Section 3.4.

The design deliberately avoids over-engineering by leveraging lightweight, well-understood technologies (PostgreSQL, REST, Docker) while retaining extensibility for instance, through pluggable runtimes or future integration with mature proxies like Envoy. This balance between simplicity and capability ensures the prototype remains feasible for implementation within the scope of this research, yet representative of real-world operational constraints.

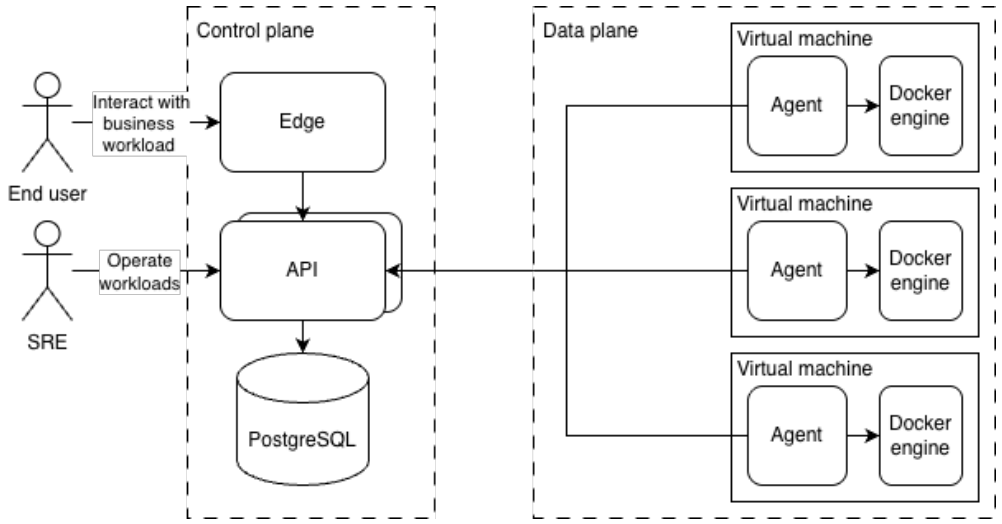

Fig. 4.3. Detailed component diagram of the finalized system architecture, illustrating interactions between the SRE client, control plane (with PostgreSQL backend), data-plane nodes (each running an agent and containerized workloads), and the Edge ingress component.

Figure 4.3 presents a comprehensive view of the finalized architecture, capturing data flows and component responsibilities. This blueprint serves as the definitive reference for the subsequent implementation phase and provides the structural foundation against which the prototype's correctness and performance will be validated.

4.2 Implementation

4.2.1 Technical Scenarios

This section presents the concrete implementation details of the system, illustrating how the pull-based orchestration model operates in practice through key technical scenarios. Each scenario demonstrates a core aspect of the system's functionality, including workload submission, agent discovery, lease management, failure recovery, and ingress routing.

Scenario 1: Workload Creation and Submission

The initial entry point for orchestrating a workload is through the command-line interface tool mctl. Operators define workload specifications in a declarative YAML format that encapsulates the container image, resource requirements, command invocation, environment variables, and optional port mappings. Figure 4.4 presents an example workload specification for a long-running HTTP server.

```yaml
image: "python:3.9"
command:
  - "python"
  - "-m"
  - "http.server"
  - "8000"
cpu: 200            # 200 millicores (0.2 CPU cores)
ram: 512            # 512 megabytes
env:
  PYTHONUNBUFFERED: "1"
container_port: 8000
```

Fig. 4.4. Example workload specification in YAML format for a Python HTTP server. The cpu field specifies resource requirements in millicores, while ram is expressed in megabytes.

The submission workflow begins when the operator executes mctl apply workload -f workload.yaml. The CLI tool parses the YAML specification and issues a POST request to the control plane's REST API endpoint /api/v1/workloads. Upon receiving the request, the control plane validates the specification, generates a unique identifier (UUIDv4) for the workload, and persists the workload record in PostgreSQL with an initial status of new. The system returns the workload identifier to the operator, enabling subsequent tracking and management operations.

Figure 4.5 illustrates the complete workload creation sequence.

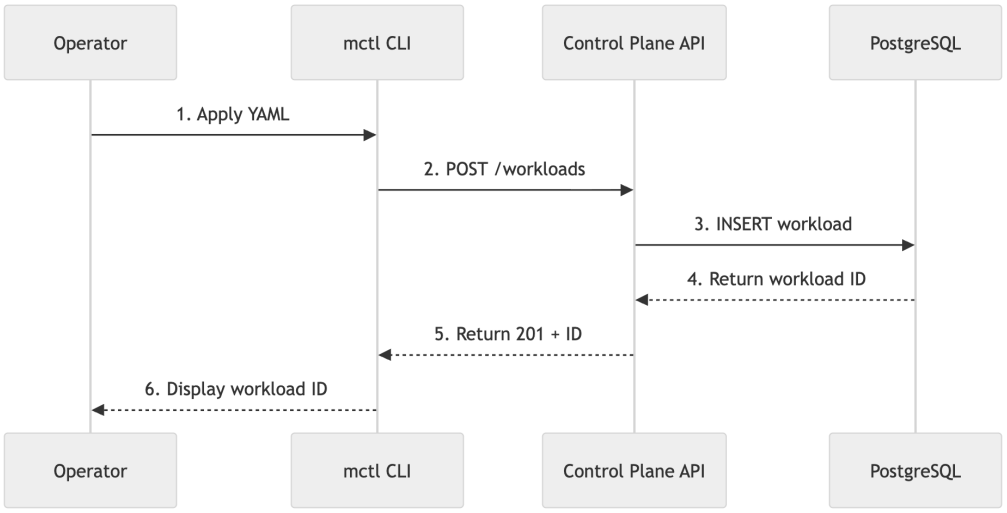

Fig. 4.5. Sequence diagram illustrating the workload creation and submission workflow. The operator submits a YAML specification via mctl, which is validated and persisted by the control plane, returning a unique workload identifier.

Scenario 2: Agent Discovery and Workload Acquisition

The pull-based model manifests most clearly in how agents discover and acquire workloads. Each agent runs a periodic reconciliation loop, orchestrated by a generic supervisor pattern that executes registered tasks at configurable intervals. By default, the supervisor triggers the workload acquisition logic every 10 seconds.

During each iteration, the agent performs the following sequence:

(1) The agent queries the control plane's $\mathrm{GET}\ /\mathrm{api}/\mathrm{v1}/\mathrm{workloads}$ endpoint, passing its available resource capacity as query parameters (e.g., $\mathrm{cpu}{=}4000$ and $\mathrm{ram}{=}8192$). The control plane responds with a list of workloads whose resource requirements fit within the agent's capacity and which do not

currently have active leases.

(2) The agent filters the response to exclude workloads it has already acquired, maintaining a local cache of owned workloads to prevent duplicate acquisition.

(3) If candidate workloads remain, the agent selects one (using a first-fit strategy) and attempts to acquire a lease by issuing a
PUT /api/v1/workloads/{id}/lease
request to the control plane. Control plane automatically recognizes node id with mTLS.

(4) The control plane records the lease in the database, associating the workload identifier with the requesting node's identifier, along with a timestamp. This mechanism is inspired by the concept of leases as fault-tolerant coordination primitives introduced by Gray and Cheriton [42], where time-bound locks enable distributed nodes to assert ownership without requiring continuous communication. The lease acquisition is atomic: if multiple agents simultaneously attempt to lease the same workload, the database's uniqueness constraint ensures only one succeeds.

Once the lease is acquired, the agent transitions the workload locally to the pending status and initiates the container creation process using the Docker Engine API.

Figure 4.6 depicts the workload acquisition sequence with lease management.

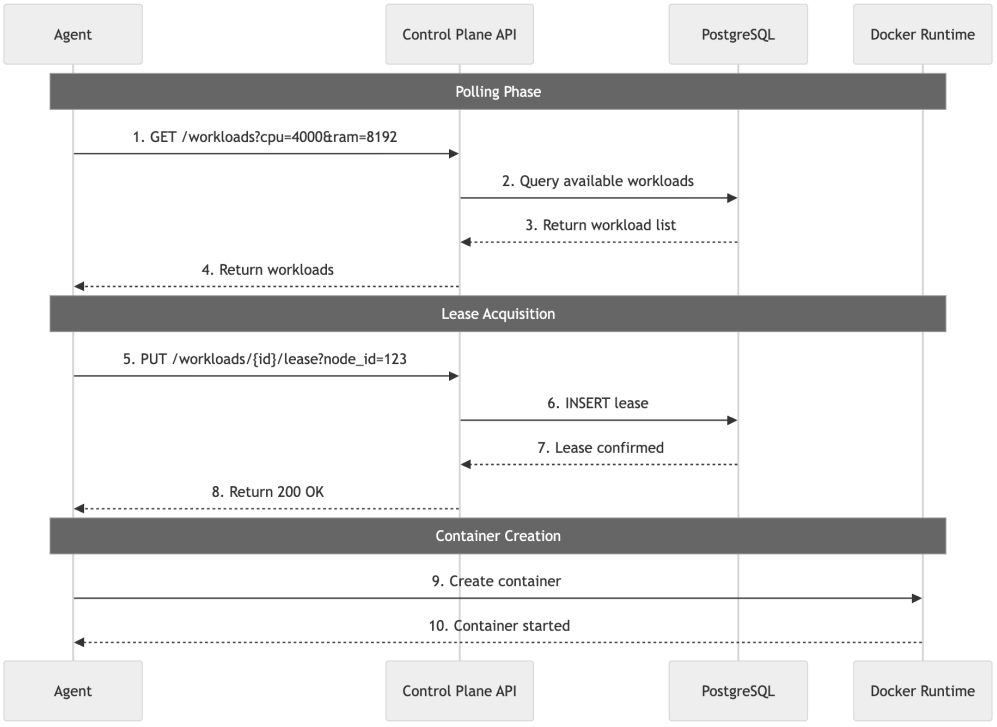

Fig. 4.6. Sequence diagram illustrating agent-driven workload acquisition. The agent polls for available workloads, acquires a lease, and creates the container via Docker.

Scenario 3: Container Lifecycle Management

Upon successfully leasing a workload, the agent invokes the container runtime to create and start the container according to the specification. The agent translates the abstract workload specification into Docker-specific configuration, setting CPU and memory limits using cgroups via Docker's $-\mathrm{cpus}$ and $-\mathrm{memory}$ flags, passing environment variables, and exposing the specified container port.

The agent then enters a monitoring phase, during which it periodically checks the container's health status. If the container exits unexpectedly, the agent attempts to restart it, subject to a configurable retry policy. Throughout the container's lifetime, the agent renews the lease every 10 seconds by issuing subsequent PUT requests to the control plane, updating the lease's $\mathrm{updated\_at}$ timestamp. This

heartbeat mechanism signals to the control plane that the workload remains under active management.

Scenario 4: Lease Expiration and Fault Tolerance

A critical aspect of the system's resilience is its handling of node failures. The control plane runs a background task at a configurable interval (default: 30 seconds) that identifies and expires stale leases. A lease is considered stale if its $\mathrm{updated\_at}$ timestamp predates a configurable threshold (default: 30 seconds). Both the lease lifetime and the expiration check interval can be tuned via command-line flags to balance detection speed against coordination overhead for a given deployment environment. When a node fails unexpectedly for example, due to a VM crash, network failure, or agent process termination it ceases to renew its leases. Consequently, the background task detects the stale leases and deletes them from the database, thereby releasing the associated workloads.

Once a workload's lease expires, the workload becomes available for acquisition by other agents. The system tracks workload status transitions based on event history: if a workload remains in the $\mathrm{pending}$ state without a valid lease for more than 90 seconds, it is automatically marked as stuck, indicating a persistent failure to start. Conversely, if a workload in the active state loses its lease, it becomes eligible for rescheduling onto a healthy node.

This design embodies the pull model's automatic recovery property: failures are detected indirectly through the absence of lease renewals, and recovery occurs automatically as other agents discover and acquire the orphaned workloads. This approach contrasts with consensus-based failure detection used in systems like ZooKeeper [24] or Chubby [25], which require quorum agreement on node

liveness. By relying on lease expiration as an implicit failure signal, the system avoids the coordination overhead inherent in such protocols. No explicit push-based notification or failure detection mechanism is required from the control plane.

Figure 4.7 illustrates the fault tolerance mechanism, including node failure detection and workload rescheduling.

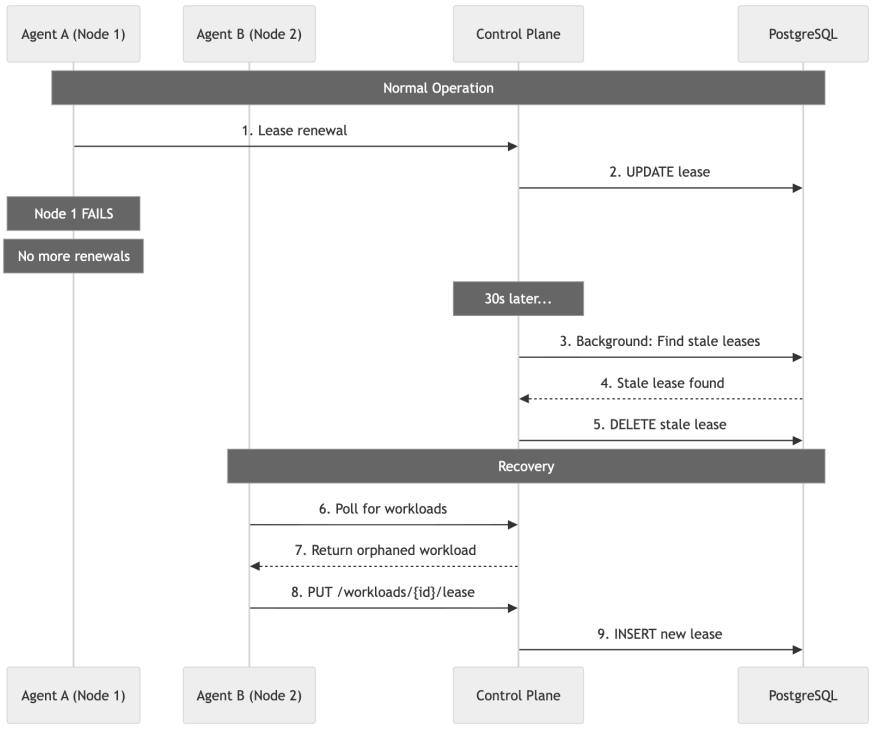

Fig. 4.7. Sequence diagram illustrating fault tolerance through lease expiration. When Agent A fails, the background task expires the stale lease after the configured timeout (default: 30 seconds), enabling Agent B to acquire and reschedule the workload.

Scenario 5: Event Propagation and Status Tracking

The system maintains an auditable history of state transitions through an event log. Agents generate events for significant lifecycle events: container creation, container exit, health check success, and health check failure. Each event

encapsulates the workload identifier, node identifier, action type, and outcome status. Agents POST these events to the /api/v1/events endpoint, where the control plane persists them in the database.

The control plane uses event history to infer workload status. For example, a sequence of successful health check events indicates a healthy active workload, while repeated exit events with non-zero exit codes suggest a failed state. This event-driven status computation ensures that the system's view of workload health derives from ground-truth observations from the data plane, rather than assumptions made by the control plane.

Figure 4.8 shows the event propagation flow from agent to control plane, including status inference.

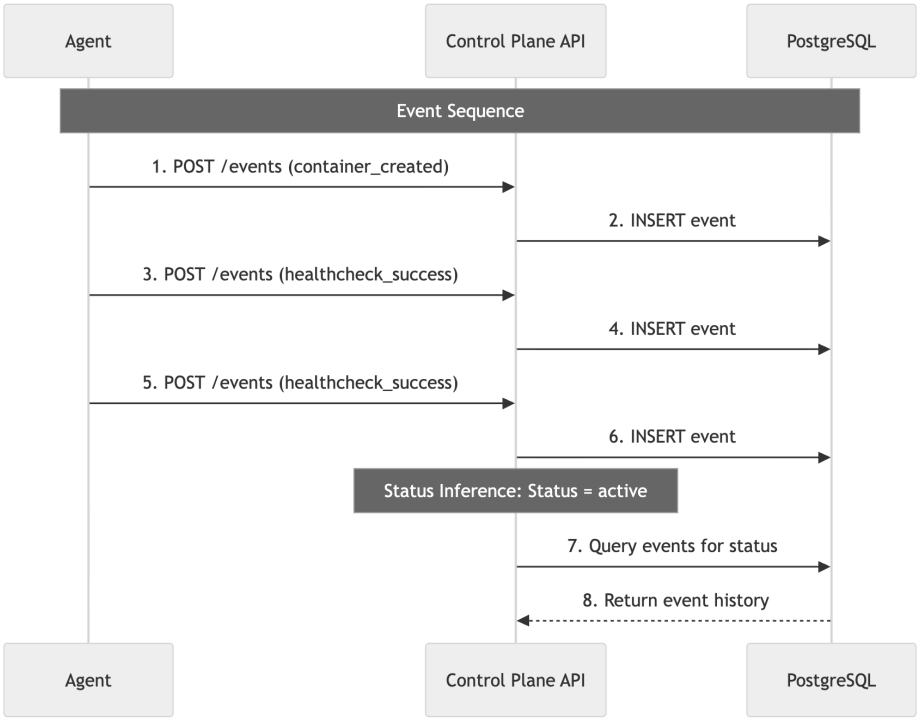

Fig. 4.8. Sequence diagram illustrating event propagation and status inference. Agents emit lifecycle events which are persisted in PostgreSQL; the control plane infers workload status from event history patterns.

Scenario 6: Ingress Routing via Edge Component

Workloads that expose HTTP endpoints require a unified access point for external clients. The Edge component addresses this need by periodically querying the control plane's GET /api/v1/edges endpoint to retrieve the current mapping of route prefixes to workload backend addresses. Each edge entry specifies a path prefix (e.g., /my-service) and an upstream address (e.g., node-1:8000).

The Edge component implements a reverse proxy that parses incoming HTTP requests, extracts the path prefix, and forwards the request to the corresponding backend using HTTP. This design enables operators to expose services through a single canonical endpoint while the actual workload instances may be scheduled on any node in the cluster. As workloads are rescheduled following failures, the Edge automatically updates its routing configuration upon the next periodic fetch, ensuring that traffic continues to flow to healthy instances without manual reconfiguration.

Implementation Architecture Summary

The implementation is organized into four main components, each adhering to a layered architecture pattern that separates concerns across domain, use case, infrastructure, and implementation layers:

- **Control Plane** (pkg/controlplane/): Stateless REST API server that exposes endpoints for workload management, lease operations, and event ingestion. Uses Uber FX for dependency injection and runs background tasks for lease expiration.

- **Agent** (pkg/agent/): Pull-based worker that periodically polls the control

plane, manages container lifecycle via Docker, renews leases, and reports events. Employs a supervisor pattern for periodic task management.

- **Edge** (pkg/edge/): Network proxy that dynamically configures routing rules based on edge configurations fetched from the control plane.
- **mctl** (pkg/mctl/): Command-line interface for workload submission, querying, and management.

All components communicate exclusively via REST over HTTPS, with the control plane maintaining PostgreSQL as the authoritative data store. The control plane's stateless compute layer enables horizontal scaling: multiple instances can run behind a load balancer without application-level consensus protocols or leader election, since no in-memory state is maintained between requests. All durable state is managed by PostgreSQL. Agents are stateless and homogeneous, and the pull-based model distributes scheduling decisions across the agent fleet rather than concentrating them in a central scheduler. The primary scaling ceiling is PostgreSQL throughput under concurrent lease operations rather than the orchestration model itself.

Figure 4.9 presents a comprehensive end-to-end sequence demonstrating the complete lifecycle from workload creation through node failure to automatic recovery.

The complete source code of the implementation is publicly available at https://github.com/wensiet/morchy.

This implementation demonstrates that a pull-based orchestration system can be constructed with minimal operational complexity while retaining the core benefits of automatic recovery, fault tolerance, and decentralized coordination.

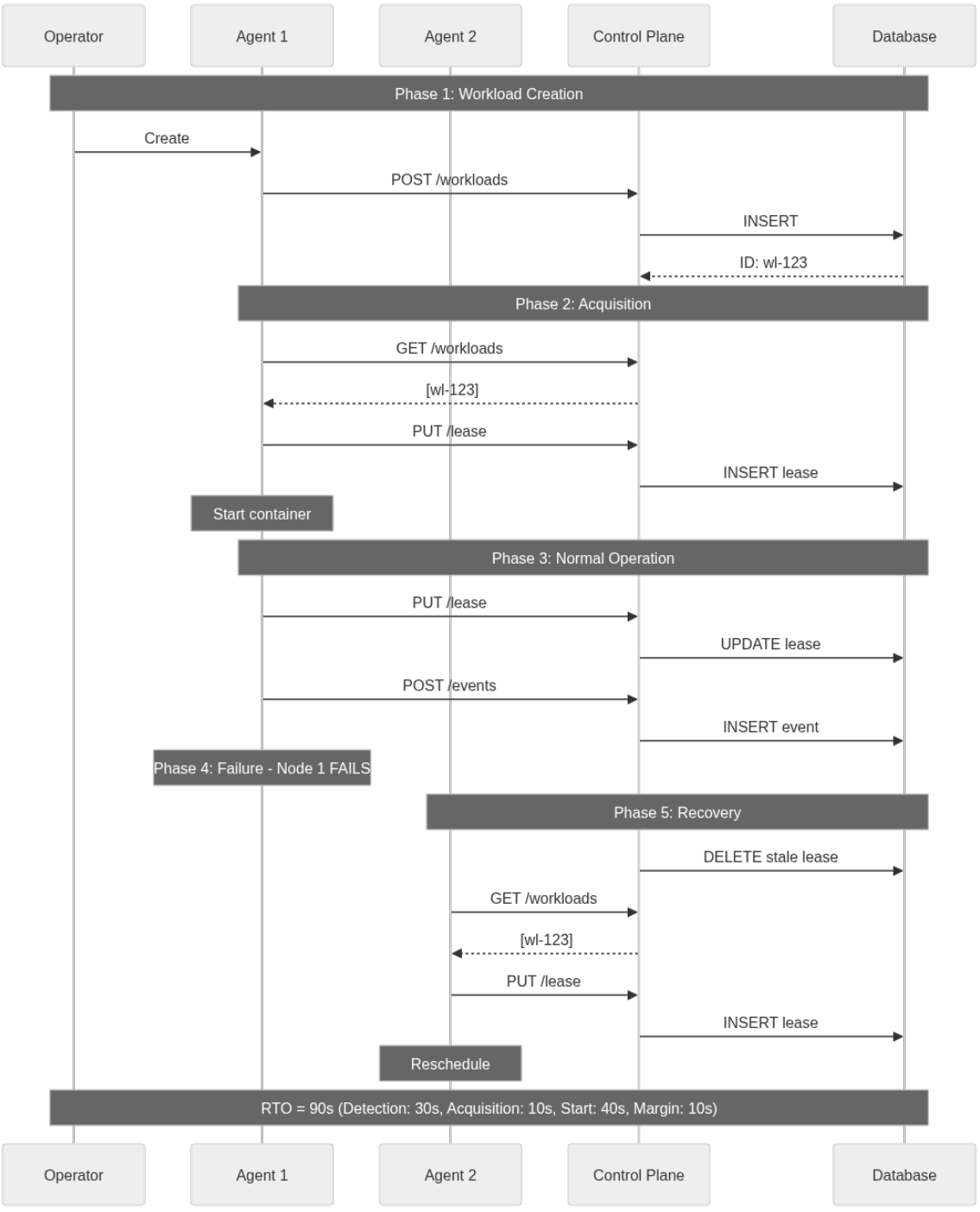

Fig. 4.9. Comprehensive end-to-end sequence demonstrating the complete pull-based orchestration lifecycle: workload creation, agent acquisition, normal operation with lease renewal, node failure detection through lease expiration, and automatic recovery by another agent. The annotated RTO box shows how the 90-second Recovery Time Objective is achieved.

The subsequent evaluation chapter presents empirical measurements of the system's performance, including recovery time objective achievement and resource efficiency.

Chapter 5

Evaluation

5.1 Methodology

5.1.1 Test Environment

The evaluation was conducted on real distributed infrastructure provided by Selectel, ensuring representative network conditions and resource constraints similar to production deployments. The test infrastructure consisted of:

- **Infrastructure Provider**: Selectel cloud platform
- **Architecture**: 3 nodes (1 control plane, 2 agents)
- **Node Configuration**: Each node configured with 2 vCPU and 4 GB RAM
- **Platform**: AMD64 architecture
- **Control Plane**: Deployed and exposed on port 8080
- **Agent Nodes**: 2 agents deployed on separate nodes, each configured with resource reservations for CPU and memory

- **Container Runtime**: Docker Engine for container lifecycle management
- **Database**: PostgreSQL 16 hosting the application schema with all migrations applied
- **Security**: Mutual TLS (mTLS) enabled on all inter-component communication channels (control plane, agents, edge), fulfilling NFR4. Client certificates were provisioned to each agent and the edge component, with the control plane enforcing certificate-based authentication.

5.1.2 Test Scenarios

Five comprehensive test scenarios were developed to validate system functionality and measure key performance metrics. To establish strong statistical validity, scenarios were executed with 100 trials each.

1. **BasicWorkloadAcquisition**: Single workload creation and scheduling to verify basic orchestration functionality
2. **MultipleWorkloads**: Two workloads created simultaneously and distributed across two agent nodes to test distributed concurrent scheduling
3. **Rescheduling**: Agent failure simulation to measure recovery time and automatic rescheduling
4. **ConcurrentWorkloads**: Two workloads created simultaneously on a single node (one agent disabled) to test single-node resource contention and concurrent container management
5. **EdgeRouting**: Workload creation with port mapping to verify network proxy configuration and edge routing functionality

All tests followed a black-box approach, interacting exclusively through the control plane API and verifying expected state transitions in the database and container runtime.

5.1.3 Metrics Definition

Two primary metrics were measured to evaluate system performance:

- **Service Level Objective (SLO)**: Time from workload creation request to the control plane marking the workload status as `active`. This metric measures how quickly the system can schedule and initialize new workloads.
- **Recovery Time Objective (RTO)**: Time from node failure (process termination) to the control plane marking the workload status as `active` on a different node. This metric measures the system's ability to recover from node failures.

The `active` status is used as the completion criterion because it indicates that the control plane has confirmed successful workload initialization, representing the end-to-end scheduling latency including lease acquisition, container creation, and status reporting.

5.2 Test Results

5.2.1 Service Level Objective (SLO)

Table 5.1 presents the SLO measurements across different workload scenarios, including percentile values and 95% confidence intervals.

Scenario	Min	Max	Avg	Median	StdDev	P90	P95	P99	CI Low	CI High	Trials
Single workload	10.5s	22.4s	14.2s	14.1s	2.0s	15.2s	19.6s	20.9s	13.8s	14.6s	100
Two workloads	11.6s	49.9s	17.9s	19.5s	4.9s	21.1s	21.4s	22.7s	17.0s	18.9s	99
Concurrent (single node)	21.4s	41.2s	31.5s	32.2s	4.1s	33.2s	40.2s	41.2s	30.7s	32.3s	100
Edge routing	14.1s	27.0s	23.6s	25.7s	4.3s	26.5s	26.6s	26.9s	22.8s	24.5s	100

TABLE 5.1
Service Level Objective (SLO) measurements with confidence intervals and percentiles

The single workload scenario demonstrates that the system can schedule individual workloads with an average time of 14.2 seconds (95% CI: [13.8s, 14.6s]). The P95 latency of 19.6 seconds indicates that 95% of workloads are scheduled within this bound, providing a reliable upper estimate for capacity planning.

When scheduling two workloads simultaneously across two nodes, the average SLO increases to 17.9 seconds (95% CI: [17.0s, 18.9s]). The P95 of 21.4 seconds shows that the typical concurrent scheduling overhead is modest, with a P99 of 22.7 seconds indicating that nearly all concurrent scheduling operations complete within this bound.

The concurrent workload scenario, where both workloads are scheduled on a single node, yields an average SLO of 31.5 seconds (95% CI: [30.7s, 32.3s]) with a P95 of 40.2 seconds. This represents a significant increase compared to the distributed two-workload scenario, reflecting the resource contention that occurs when a single agent must manage multiple container lifecycles simultaneously. The standard deviation of 4.1 seconds indicates consistent performance despite the contention.

A Mann-Whitney U test confirms that the difference between single and two-workload SLO distributions is statistically significant ($U = 2749.0$, $p < 0.001$). A second Mann-Whitney U test comparing the two-workload distributed scenario

with the single-node concurrent scenario yields $U = 110.0$ ($p < 0.001$), confirming that single-node resource contention introduces a statistically significant overhead compared to distributed scheduling.

Figure 5.1 visualizes the distribution of SLO measurements across scenarios, highlighting the concentration of values below 25 seconds for distributed scenarios and the higher latency range (21–41 seconds) for single-node concurrent scheduling.

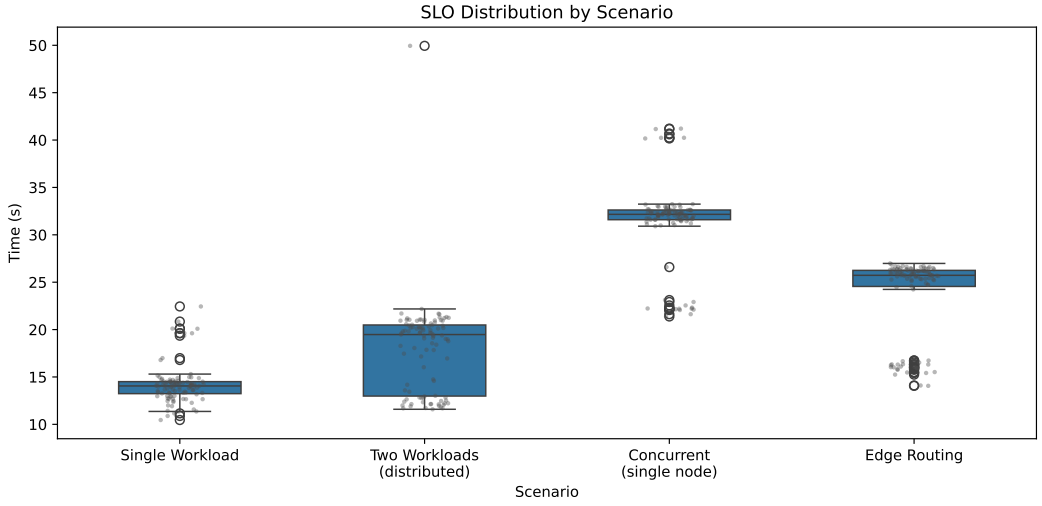

Fig. 5.1. SLO distribution by scenario. Box plots show median, quartiles, and whiskers for single workload, two-workload distributed, concurrent single-node, and edge routing scenarios. Individual data points are shown as scatter overlay.

5.2.2 Recovery Time Objective (RTO)

Table 5.2 presents the RTO measurements from node failure scenarios.

Metric	Min	Max	Avg	Median	StdDev	P90	P95	P99	CI Low	CI High	Trials
RTO (node failure)	27.0s	66.4s	47.8s	48.9s	10.2s	57.9s	59.3s	60.0s	45.8s	49.8s	100

TABLE 5.2
Recovery Time Objective (RTO) measurements with confidence intervals and percentiles

The RTO measurements demonstrate that the system can recover from node failures with an average time of 47.8 seconds (95% CI: [45.8s, 49.8s]). The narrow confidence interval ($\pm$2.0 seconds) indicates highly predictable recovery behavior. The P95 of 59.3 seconds provides a reliable upper bound: 95% of node failures result in workload recovery within one minute.

The RTO is composed of three phases: (1) lease expiration detection (up to the configured timeout, 30 seconds by default, depending on the alignment between the background task execution and the last lease renewal), (2) backup agent polling and lease acquisition (2–20 seconds), and (3) container creation and status reporting (approximately 10–15 seconds). The standard deviation of 10.2 seconds indicates moderate variance in recovery times, primarily driven by differences in container startup times and polling alignment.

All 100 trials resulted in successful workload recovery, demonstrating that the lease-based failure detection mechanism operates reliably.

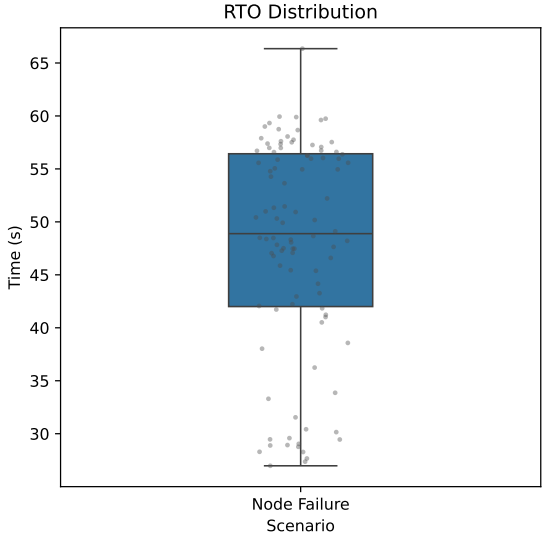

Fig. 5.2. RTO distribution for node failure recovery. The box plot shows median, quartiles, and whiskers. Individual data points are shown as scatter overlay.

Figure 5.3 presents the cumulative distribution function for all measured

metrics, showing that SLO values converge rapidly while RTO exhibits a more gradual slope due to the configurable 30-second lease timeout component.

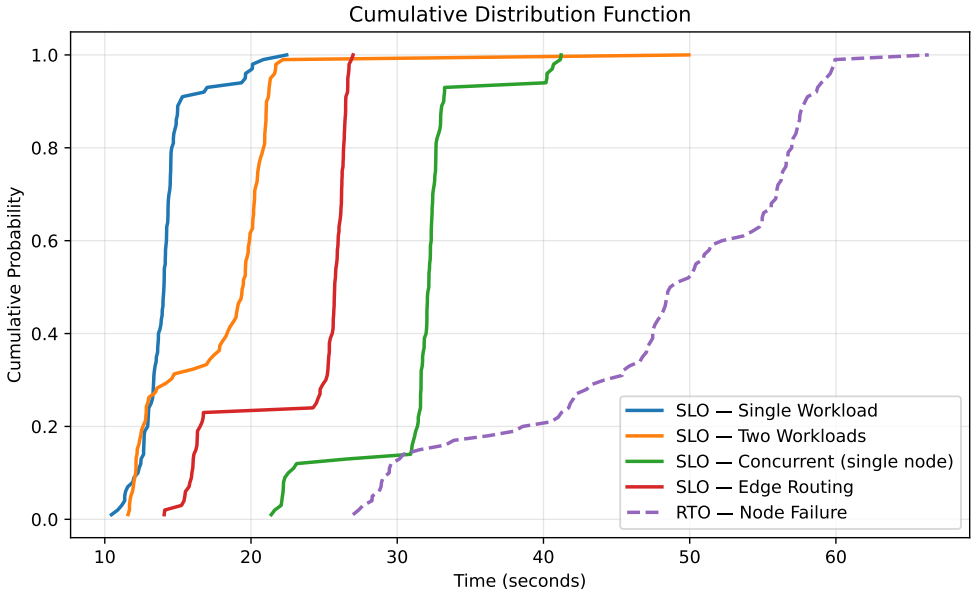

Fig. 5.3. Cumulative Distribution Function (CDF) of SLO and RTO measurements. The CDF shows the probability that a measurement falls at or below a given time value.

5.2.3 Functional Validation

Beyond quantitative metrics, the tests validated core functional requirements:

- **Distributed Operation**: Multiple agents successfully coordinated workload assignment without conflicts. The pull-based model with lease exclusivity prevented duplicate workload acquisitions across agents.
- **Automatic Rescheduling**: All 100 trials of the rescheduling scenario resulted in successful workload recovery on a different node without manual intervention. The lease expiration mechanism correctly detected node failures and triggered re-acquisition by backup agents.

- **Concurrent Workload Handling**: The system successfully handled multiple simultaneous workload creations without resource exhaustion or race conditions. All 100 concurrent workload trials (200 individual workloads on a single node) completed successfully, demonstrating that a single agent can manage multiple container lifecycles concurrently. Database transaction isolation ensured correct serialization of concurrent lease acquisition requests.

- **Lease Exclusivity**: Each workload was acquired by exactly one agent, with no double-acquisition or lease conflicts observed across 700 total workload placements across all test scenarios.

- **Secure Communication**: All test scenarios were executed with mTLS enabled, confirming that encrypted, mutually authenticated communication between agents, the control plane, and the edge component operates correctly under load without observable latency degradation.

5.3 Discussion

5.3.1 Performance Analysis

The test results demonstrate that the pull-based orchestration model achieves predictable scheduling performance with acceptable latency for short-lived workloads. The average SLO of 14.2 seconds (95% CI: [13.8s, 14.6s]) for single workloads, 17.9 seconds (95% CI: [17.0s, 18.9s]) for distributed two-workload scheduling, and 31.5 seconds (95% CI: [30.7s, 32.3s]) for single-node concurrent scheduling is suitable for batch processing, CI/CD pipelines, and other use cases

where workload initialization time is not critical compared to total execution time.

The RTO measurements confirm that the lease-based failure detection mechanism provides reliable fault tolerance with a tight confidence interval (95% CI: [45.8s, 49.8s]). While the default 30-second lease timeout introduces a lower bound on failure detection, this value is configurable and the trade-off simplifies the architecture by eliminating complex heartbeat protocols and consensus algorithms.

The standard deviation values (2.0 seconds for single SLO, 4.1 seconds for concurrent SLO, 10.2 seconds for RTO) indicate low to moderate variance. The deterministic nature of the lease expiration mechanism ensures consistent failure detection performance, contributing to the overall predictability of the system.

5.3.2 Resource Efficiency

The control plane was configured with 0.5 vCPU and 1 GB RAM resource limits, representing a minimal footprint compared to existing orchestration platforms. This configuration successfully handled the tested workload (up to 2 concurrent workloads per node) without performance degradation or resource exhaustion.

Table 5.3 compares the control plane resource requirements of the proposed system with Kubernetes and HashiCorp Nomad.

Even in a comparable HA configuration (three control plane replicas behind a load balancer), the proposed system requires significantly fewer resources than both Kubernetes and Nomad. This advantage stems from the control plane's stateless compute layer: because the application holds no in-memory cluster state and delegates persistence to PostgreSQL, replicas are interchangeable and require no

Platform	Control Plane CPU	Control Plane Memory
Kubernetes (HA)	13 vCPU	18.5 GB
Nomad (HA)	3 vCPU	1.5–2.25 GB
Proposed System (HA)	1.75 vCPU	3.25 GB
Proposed System (single)	0.5 vCPU	1 GB

TABLE 5.3
Control plane resource requirements by deployment configuration. HA configurations assume replicated control plane instances with a load balancer. Proposed System HA figures assume three application replicas (0.5 vCPU / 1 GB each) behind a lightweight load balancer (∼0.25 vCPU / 256 MB). Kubernetes data from [1], Nomad data from [2].

application-level consensus protocol or leader election among themselves. Benchmarks of lightweight Kubernetes distributions on edge hardware [33], [34] report similar overhead patterns, confirming that eliminating the etcd dependency and scheduler loop yields measurable resource advantages even when HA replication is factored in.

5.3.3 Design Trade-offs

The evaluation reveals fundamental trade-offs inherent in the pull-based orchestration model:

Simplicity vs. Feature Richness: The minimalist design yields substantial resource efficiency gains and operational simplicity, but sacrifices advanced features such as sophisticated scheduling algorithms (bin-packing, affinity/anti-affinity), rolling updates, and comprehensive observability. For organizations with simple orchestration requirements, this trade-off is favorable; for complex enterprise deployments, the limitations may be prohibitive.

Detection Latency vs. Complexity: The default 30-second lease timeout

introduces bounded failure detection latency, but eliminates the need for complex heartbeat protocols and consensus-based failure detection. This trade-off is acceptable for use cases where rapid failure detection is not critical, but may be insufficient for mission-critical applications requiring sub-second recovery times. Operators can reduce the timeout at the cost of increased false-positive expirations under transient network delays.

Scheduling Optimality vs. Scalability: The first-come-first-served scheduling algorithm may yield suboptimal workload placement compared to global optimization algorithms such as those employed by Firmament [19] or TetriSched [44], but enables linear scalability because scheduling work is distributed across agents rather than centralized in the control plane. As the number of agents increases, aggregate polling frequency increases proportionally, and the control plane's stateless compute layer can be replicated horizontally behind a load balancer without coordination overhead. The system's scaling ceiling is therefore determined by PostgreSQL throughput rather than by the orchestration model itself. Hawk [18] demonstrated a similar trade-off by delegating short tasks to distributed schedulers, and Sparrow [16] showed that stateless random sampling scheduling achieves competitive latency for short-lived tasks without global state.

Distributed vs. Single-Node Scheduling: The concurrent workload results reveal that single-node scheduling introduces a 76% overhead compared to distributed scheduling (31.5s vs. 17.9s average SLO). This overhead arises because a single agent must sequentially handle container creation for multiple workloads while also refreshing leases and reconciling state. Distributing workloads across multiple agents mitigates this bottleneck, as each agent independently manages its subset of containers. This finding suggests that the system benefits from over-provisioning agent nodes to avoid resource contention, particularly when

workload density per node is high.

5.3.4 Threats to Validity

Several limitations affect the generalizability of the evaluation findings:

Scale Limitations: The evaluation tested up to 2 concurrent workloads per node across 2 agents with 100 trials per scenario. While this provides strong statistical significance for the tested scale (confirmed by narrow 95% confidence intervals), large-scale deployments with hundreds of workloads and dozens of agents may exhibit different performance characteristics. The concurrent scenario tested only 2 workloads on a single node; higher concurrency levels (5, 10, or more workloads per node) may exhibit non-linear degradation due to resource contention. Database query performance and connection pool exhaustion become concerns at higher scale.

Workload Simplicity: Test workloads used minimal images (busybox) with simple command execution. Real-world applications with complex initialization logic, dependency resolution, and health check endpoints may exhibit longer startup times and higher resource consumption. Emerging ML-based orchestration approaches [52] suggest that adaptive scheduling could mitigate such variability, though this remains outside the scope of the current prototype.

Network Homogeneity: All nodes were deployed in the same Selectel cloud region with low-latency networking. Geographically distributed deployments with higher network latency may exhibit different performance characteristics, particularly for RTO measurements. Wang et al. [39] and Böhm and Wirtz [35] identify network heterogeneity as a primary challenge for orchestration in edge and fog environments.

Test Duration: Each trial represents a single workflow execution. Long-running tests (hours or days) would be required to detect memory leaks, resource exhaustion, and other emergent behaviors that may not be apparent in short-duration tests.

Absence of Comparative Baseline: The evaluation measures the proposed system in isolation. However, the observed scheduling and recovery performance is comparable to published figures for existing platforms: Kubernetes reports pod startup SLOs of under 5 seconds for scheduling decisions alone with cached images [1], with default node failure recovery requiring approximately 6–10 minutes (node-monitor-grace-period of 40 seconds plus pod-eviction-timeout of 5 minutes), though this can be reduced to approximately 45–95 seconds with tuned parameters. HashiCorp Nomad achieves workload recovery in approximately 15–50 seconds in small clusters through heartbeat-based failure detection with a default grace period of 10 seconds [2]. The proposed system's RTO of 47.8 seconds is comparable to both Nomad and tuned Kubernetes, while achieving this without requiring parameter tuning or active health-check configuration. Its SLO of 14.2 seconds includes the full end-to-end path from API call through polling, lease acquisition, container creation, and status confirmation. A rigorous comparative study deploying equivalent workloads on K3s or Nomad under the same infrastructure would strengthen the empirical claims but is left as future work.

5.4 Conclusion

This chapter presented empirical evaluation of the pull-based orchestration system, demonstrating predictable scheduling performance (SLO of 14.2 seconds

average for single workloads, 95% CI: [13.8s, 14.6s]), reliable fault tolerance (RTO of 47.8 seconds average, 95% CI: [45.8s, 49.8s]), and measurable single-node contention overhead (concurrent SLO of 31.5 seconds, 95% CI: [30.7s, 32.3s]) on real distributed infrastructure. Statistical analysis confirmed that both distributed concurrent scheduling and single-node contention introduce measurable overhead (Mann-Whitney U tests, $p < 0.001$). Functional testing validated distributed operation, automatic rescheduling, and concurrent workload handling across 700 total workload placements.

The evaluation confirms that the pull-based architecture achieves substantial resource efficiency gains even in comparable HA configurations (Table 5.3) while maintaining acceptable performance for short-lived workloads. The minimalist design successfully trades feature richness for operational simplicity, making the system suitable for resource-constrained environments and use cases where Kubernetes operational complexity is disproportionate to requirements.

The test results provide evidence that pull-based orchestration is a viable approach for small-scale deployments, offering a compelling alternative for organizations seeking simplicity over feature completeness. Future work should address scalability testing with larger workload counts, long-running stability tests, and performance characterization under network latency and partition scenarios.

Chapter 6

Conclusion

This thesis set out to explore whether a pull-based orchestration model could offer a simpler, lighter alternative to platforms like Kubernetes and Nomad, without giving up the fault tolerance that distributed workloads demand. Through the Design Science Research methodology, the work moved from problem analysis to a working prototype and finally to empirical evaluation on real infrastructure. The results, while limited in scale, suggest that the approach has merit for the right kind of workload.

6.1 Summary of Findings

Chapter 1 posed two research questions. The first asked whether a pull-based system can meaningfully reduce operational overhead while still recovering from failures quickly. The evaluation in Chapter 5 gives a reasonably clear answer: the control plane runs on 0.5 vCPU and 1 GB of RAM, significantly less than a highly available Kubernetes setup (see Table 5.3), yet it recovers from node failures in under 48 seconds on average, well within the 90-second target established in

Chapter 3. Across 700 workload placements in five test scenarios, not a single lease conflict or duplicate acquisition occurred, which speaks to the correctness of the coordination mechanism.

The second research question concerned the architectural trade-offs inherent in the pull-based orchestration model. What emerges from the analysis is a consistent pattern of trading sophistication for simplicity. Because the control plane's compute layer is stateless (persistence is delegated to PostgreSQL) and agents pull work on their own schedule, there is no need for application-level consensus protocols or cluster state replication, but this also means the scheduler is limited to first-come-first-served placement. It cannot make globally optimal decisions the way systems like Firmament [19] or TetriSched [44] can. The default 30-second lease timeout keeps failure detection simple, but it also sets a configurable floor on how quickly the system can react to outages. As discussed in Chapter 3, the design favors availability over strong consistency, which is a reasonable choice for short-lived batch workloads where a brief scheduling delay is tolerable. One finding worth noting is that scheduling multiple workloads on a single node is roughly 76% slower than distributing them. The agent has to handle container creation sequentially while also refreshing leases and reconciling state, so the system clearly works better when workloads are spread across nodes.

6.2 Strengths, Limitations, and Applicability

The main advantage of the proposed system is its simplicity. The pull model sidesteps several failure modes that plague push-based architectures: broken deliveries after network partitions, duplicate scheduling from stale controller state, and the operational burden of maintaining a complex control plane. The lease

mechanism itself is well-understood and straightforward, drawing on established distributed systems principles [42]. Security was also kept in scope: all communication between components uses mTLS, so the system does not cut corners on that front despite its minimalism.

These properties make the system a reasonable fit for a handful of practical scenarios. Short-lived batch jobs and CI/CD pipeline workloads, where the time spent starting a container is small compared to the time it runs. Edge and IoT deployments, where the hardware simply cannot accommodate the resource demands of a full Kubernetes stack. Small teams and organizations that need orchestration but cannot justify hiring dedicated platform engineers. The business scenario from Chapter 3, a B2B authentication service recovering from infrastructure failures, is itself a concrete example of where this approach would be appropriate.

That said, the limitations are real and should not be understated. The evaluation covered only two agents with at most two concurrent workloads per node, and it would be premature to assume the same behavior at larger scales. Database contention and polling overhead may become bottlenecks that were not visible here. The system lacks features that many production environments consider essential: rolling updates, autoscaling, affinity rules, and sophisticated placement strategies. Test workloads used minimal container images, so the scheduling times measured may not reflect how real applications behave. All testing was done within a single cloud region, meaning the effects of network latency in geographically distributed setups remain unknown.

6.3 Future Work

Several directions deserve attention. The most immediate is testing at larger scale, with dozens of agents and hundreds of workloads, to see whether the linear scaling properties hold under real production load. Long-running tests over days or weeks would help surface memory leaks and resource exhaustion issues that short trials cannot catch. Investigating optimal lease timeout values for different deployment environments and workload profiles would help operators tune the trade-off between detection speed and coordination overhead.

On the feature side, node selector rules would allow workloads to express hardware or topology requirements. For instance, a machine learning training job might require a GPU, and the scheduler should only consider agents that satisfy that constraint. Rolling workload updates, similar to what Kubernetes provides for deployments, would enable operators to update container images or configuration without downtime by progressively replacing old instances with new ones. A higher-level abstraction on top of individual workloads, analogous to ReplicaSets in Kubernetes, would let users declare a desired number of replicas and let the system handle creation, scaling, and reconciliation automatically. Finally, integrated telemetry, including metrics export, structured logging, and distributed tracing, would give operators visibility into scheduling decisions, agent health, and workload lifecycle events, which is essential for running any orchestration system in production.

Bibliography cited

[1] Kubernetes Authors, *Kubernetes documentation: Architecture overview*, https://kubernetes.io/docs/concepts/architecture/, Accessed: 31 October 2025, 2025.

[2] HashiCorp, *Nomad documentation: Architecture overview*, https://developer.hashicorp.com/nomad/docs/architecture, Accessed: 31 October 2025, 2025.

[3] M. Sebrechts, "Orchestrator conversation: Distributed management of cloud applications," *International Journal of Network Management*, 2018.

[4] D. Palma, M. Rutkowski, and T. Spatzier, "Tosca simple profile in yaml version 1.0," OASIS, Committee Specification Draft 04 / Public Review Draft 01, 2015.

[5] P. Rauba and N. Sontakos, "Self-healing machine learning: A framework for autonomous adaptation in real-world environments," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, Top-tier venue (NeurIPS), Author H-index: 5, 99 total citations, Curran Associates, 2024.

[6] R. K. Vankayalapati, *Zero-trust security models for cloud data analytics: Enhancing privacy in distributed systems*, https://papers.ssrn.com/sol3/

papers.cfm?abstract_id=5121185, SSRN 5121185, 12 citations, Author total citations: 628 (Strong H-index), 2025.

[7] A. Dresch, "Design science research," 2014, [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-07374-3_4.

[8] C. Carrión, "Kubernetes scheduling: Taxonomy, ongoing issues and challenges," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–36, 2022. DOI: 10.1145/3539606.

[9] K. S. et al., "A survey of kubernetes scheduling algorithms," *Journal of Cloud Computing*, vol. 12, pp. 87–87, 2023. DOI: 10.1186/s13677-023-00471-1.

[10] E. Song *et al.*, "Canal mesh: A cloud-scale sidecar-free multi-tenant service mesh architecture," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM, Aug. 2024. DOI: 10.1145/3651890.3672221. [Online]. Available: %5Curl%7Bhttps://dl.acm.org/doi/10.1145/3651890.3672221%7D.

[11] M. Pantazoglou, G. Tzortzakis, and A. Delis, "Decentralized and energy-efficient workload management in enterprise clouds," *IEEE Transactions on Cloud Computing*, 2015.

[12] A. V. et al., "Large-scale cluster management at google with borg," in *Proceedings of the Tenth European Conference on Computer Systems*, ACM, 2015, pp. 1–17. DOI: 10.1145/2741948.2741964.

[13] M. S. et al., "Omega: Flexible, scalable schedulers for large compute clusters," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, ACM, 2013, pp. 351–364.

[14] B. H. et al., "Mesos: A platform for fine-grained resource sharing in the data center," in *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation*, USENIX Association, 2011, pp. 295–308.

[15] V. K. V. et al., "Apache hadoop yarn: Yet another resource negotiator," in *Proceedings of the 4th ACM Symposium on Cloud Computing*, ACM, 2013, pp. 1–16. DOI: 10.1145/2523616.2523633.

[16] K. O. et al., "Sparrow: Distributed, low latency scheduling," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, ACM, 2013, pp. 69–84.

[17] E. B. et al., "Apollo: Scalable and coordinated scheduling for cloud-scale computing," in *Proceedings of the 25th ACM Symposium on Operating Systems Principles*, ACM, 2015, pp. 91–105.

[18] P. D. et al., "Hawk: Hybrid datacenter scheduling," in *Proceedings of the USENIX Annual Technical Conference*, USENIX Association, 2015, pp. 499–510.

[19] I. G. et al., "Firmament: Fast, centralized cluster scheduling at scale," in *Proceedings of the 11th European Conference on Computer Systems*, ACM, 2016, pp. 1–16.

[20] S. Akbari, M. Hauswirth, *et al.*, "Hiku: Pull-based scheduling for serverless computing," in *Proceedings of the IEEE 25th International Symposium on Cluster, Cloud and Internet Computing*, IEEE, 2025. DOI: 10.1109/CCGRID64434.2025.00034. [Online]. Available: %5Curl%7Bhttps://arxiv.org/abs/2502.15534%7D.

[21] E. Brewer, "Cap twelve years later: How the rules have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23–29, 2012. DOI: 10.1109/MC.2012.37.

[22] S. Gilbert and N. Lynch, "Perspectives on the cap theorem," *IEEE Computer*, vol. 45, no. 2, pp. 30–36, 2012. DOI: 10.1109/MC.2011.389.

[23] G. D. et al., "Dynamo: Amazon's highly available key-value store," in *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, ACM, 2007, pp. 205–220. DOI: 10.1145/1323293.1294281.

[24] P. H. et al., "Zookeeper: Wait-free coordination for internet-scale systems," in *Proceedings of the USENIX Annual Technical Conference*, USENIX Association, 2010.

[25] M. Burrows, "The chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2006, pp. 335–350.

[26] L. Lamport, "Paxos made simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, 2001.

[27] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the USENIX Annual Technical Conference*, USENIX Association, 2014, pp. 305–319.

[28] Y. Wu *et al.*, "Task scheduling in geo-distributed computing: A survey," *IEEE Internet Computing*, vol. 36, pp. 2073–2088, 2025. DOI: 10.1109/TPDS.2025.3591010. [Online]. Available: %5Curl%7Bhttps://ieeexplore.ieee.org/document/11086502/%7D.

[29] Google, *How google protects its production services*, https://cloud.google.com/docs/security/production-services-protection, Accessed: 31 October 2025, 2025.

[30] M. Liaqat, "Federated cloud resource management: Review and discussion," *Elsevier*, 2017.

[31] D. Velev and P. Zlateva, "Cloud infrastructure security," *Open Research Problems in Network Security*, 2011.

[32] H. Rasheed, "Data and infrastructure security auditing in cloud computing environments," *Elsevier*, 2014.

[33] I. C. et al., "Performance evaluation of container orchestration tools in edge computing environments," *Sensors*, vol. 23, no. 8, p. 4008, 2023. DOI: 10.3390/s23084008.

[34] M. U. et al., "Performance analysis of lightweight container orchestration platforms for edge-based iot applications," in *Proceedings of the IEEE/ACM Symposium on Edge Computing*, IEEE, 2024, pp. 1–8. DOI: 10.1109/SEC62691.2024.00032.

[35] S. Böhm and G. Wirtz, "Cloud-edge orchestration for smart cities: A review of kubernetes-based orchestration architectures," *EAI Endorsed Transactions on Smart Cities*, vol. 6, no. 18, 2022.

[36] Q. D. et al., "Fogbus2: A lightweight and distributed container-based framework for integration of iot-enabled systems with edge and cloud computing," in *Proceedings of the International Workshop on Big Data Analytics for Edge/Fog Computing*, ACM, 2021, pp. 1–8. DOI: 10.1145/3460866.3461768.

[37] S. A. et al., "Lightweight edge slice orchestration framework," in *Proceedings of the IEEE International Conference on Communications*, IEEE, 2022, pp. 865–870. DOI: 10.1109/ICC45855.2022.9838854.

[38] F. G. et al., "A fuzzy controller for self-adaptive lightweight edge container orchestration," in *Proceedings of the 10th International Conference on Cloud Computing and Services Science*, SCITEPRESS, 2020, pp. 79–90.

[39] Z. W. et al., "Container orchestration in edge and fog computing environments for real-time iot applications," in *Computational Intelligence and Intelligent Systems*, Springer, 2022, pp. 1–21.

[40] G. M. et al., "Consensus-based distributed orchestration framework for microservices in edge computing clusters," *Future Generation Computer Systems*, 2025.

[41] Z. Bakhshi and G. Rodriguez-Navas, "Fault-tolerant permanent storage for container-based fog architectures," in *Proceedings of the IEEE Symposium on Computers and Communications*, IEEE, 2021, pp. 722–729. DOI: 10.1109/ICIT46573.2021.9453473.

[42] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proceedings of the 12th ACM Symposium on Operating Systems Principles*, ACM, 1989, pp. 202–210. DOI: 10.1145/74850.74870.

[43] J. Dean and S. Ghemawat, "Mapreduce: Simplified data processing on large clusters," in *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2004, pp. 10–10.

[44] A. T. et al., "Tetrisched: Global rescheduling with adaptive plan-ahead in dynamic heterogeneous clusters," in *Proceedings of the 11th European Conference on Computer Systems*, ACM, 2016, pp. 1–16. DOI: 10.1145/2901318.2901355.

[45] B. Burns, "Borg, omega, and kubernetes," *Communications of the ACM*, 2016.

[46] B. Burns, *Kubernetes: Up & Running 3rd edition*. O Reilly, 2022.

[47] K3S Maintainers, *K3s documentation*, https://docs.k3s.io/, Accessed: 8 January 2026, 2025.

[48] K0S Maintainers, *K0s documentation*, https://docs.k0sproject.io/, Accessed: 8 January 2026, 2025.

[49] D. job editors. "Sre engineers salaries at 2025." (2025), [Online]. Available: https://dreamjob.ru/salary/sre-inzhener (visited on 01/08/2026).

[50] Stackoverflow Maintainers, *Stack overflow developer survery 2025*, https://survey.stackoverflow.co/2025/, Accessed: 8 January 2026, 2025.

[51] Randy Bias, *The history of pets vs cattle and how to use the analogy properly*, https://cloudscaling.com/blog/cloud-computing/the-history-of-pets-vs-cattle/, Accessed: 10 January 2026, 2016.

[52] Z. Z. et al., "Machine learning-based orchestration of containers: A taxonomy and future directions," *ACM Computing Surveys*, vol. 54, no. 10s, 2022. DOI: 10.1145/3510415.

Appendix A

Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Appendix B

Even More Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.